Example to Generate Constraints

S ::= X | Y
X ::= **b** | S Y
Y ::= Z X **b** | Y **b**
Z ::= ε | **a**

S' = X' U Y'
X' = {b} U S'
Y' = Z' U **X'** U Y'
Z' = {a}

terminals: **a**,**b**
non-terminals: S, X, Y, Z

reachable (from S): S, X, Y, Z
productive: X, Z, S, Y
nullable: Z

These constraints are recursive.
How to solve them?

S', X', Y', Z' ⊆ {a,b}

How many candidate solutions

- in this case?
- for k tokens, n nonterminals?

Iterative Solution of **first** Constraints

	S'	X'	Y'	Z'
1.	{}	{}	{}	{}
2.	{}	{b}	{b}	{a}
3.	{b}	{b}	{a,b}	{a}
4.	{a,b}	{a,b}	{a,b}	{a}
5.	{a,b}	{a,b}	{a,b}	{a}

$$S' = X' \cup Y'$$
$$X' = \{b\} \cup S'$$
$$Y' = Z' \cup \mathbf{X'} \cup Y'$$
$$Z' = \{a\}$$

- Start from all sets empty.
- Evaluate right-hand side and assign it to left-hand side.
- Repeat until it stabilizes.

Sets grow in each step

- initially they are empty, so they can only grow
- if sets grow, the RHS grows (U is monotonic), and so does LHS
- they cannot grow forever: in the worst case contain all tokens

Constraints for Computing Nullable

- Non-terminal is nullable if it can derive ε

S ::= X | Y
X ::= **b** | S Y
Y ::= Z X **b** | Y **b**
Z ::= ε | **a**

S' = X' | Y'
X' = 0 | (S' & Y')
Y' = (Z' & X' & 0) | (Y' & 0)
Z' = 1 | 0

$Y' = 0$

S', X', Y', Z' ∈ {0,1}
0 - not nullable
1 - nullable
| - disjunction
& - conjunction

	S'	X'	Y'	Z'
1.	0	0	0	0
2.	0	0	0	1
3.	0	0	0	1

again monotonically growing

Computing first and nullable

- Given any grammar we can compute
  - for each non-terminal X whether nullable(X)
  - using this, the set first(X) for each non-terminal X
- General approach:
  - generate constraints over finite domains, following the structure of each rule
  - solve the constraints iteratively
    - start from least elements
    - keep evaluating RHS and re-assigning the value to LHS
    - stop when there is no more change

Summary: Algorithm for **nullable**

```
nullable = {}
changed = true
while changed do
  changed = false
  for each non-terminal X
    if ((X is not nullable) and
        (grammar contains rule    X ::= ε | ...    )
          or  (grammar contains rule    X ::= Y1 ... Yn | ...
        where {Y1,...,Yn} ⊆ nullable)
    then
      nullable = nullable U {X}
      changed = true
```

Summary: Algorithm for **first**

for each nonterminal X: first(X)={}

for each terminal t: first(t)={t}

repeat

for each grammar rule X ::= Y(1) … Y(k)

for i = 1 to k **do**

if i=1 or {Y(1),…,Y(i-1)} ⊆ nullable **then**

first(X) = first(X) U first(Y(i))

until none of first(…) changed in last iteration

Y_{i-1}

Y_i

Y_1 … Y_k

Follow sets. LL(1) Parsing Table

Exercise Introducing Follow Sets

Compute nullable, first for this grammar:

stmtList ::= ε | stmt stmtList

stmt ::= assign | block

assign ::= **ID** **=** **ID** **;**

block ::= **beginof** **ID** stmtList **ID ends**

def fun(...)

Describe a parser for this grammar and explain how it behaves on this input:

fun end

```
beginof myPrettyCode
    x = u;
    y = v;
myPrettyCode ends
```

How does a recursive descent parser look like?

```
def stmtList =
  if (???) {}
  else { stmt; stmtList }
def stmt =
  if (lex.token == ID) assign
  else if (lex.token == beginof) block
  else error("Syntax error: expected ID or beginonf")
...
def block =
  { skip(beginof); skip(ID); stmtList; skip(ID); skip(ends) }
```

what should the condition be?

Problem Identified

stmtList ::= ε | stmt stmtList

stmt ::= assign | block

assign ::= **ID** = **ID** ;

block ::= **beginof** **ID** stmtList **ID** **ends**

Problem parsing stmtList:

- **ID** could start alternative stmt stmtList
- **ID** could **follow** stmt, so we may wish to parse **ε** that is, do nothing and return

- For nullable non-terminals, we must also compute what **follows** them

LL(1) Grammar - good for building recursive descent parsers

$X := \varepsilon \mid \ldots$

- Grammar is LL(1) if for each nonterminal X
  - first sets of different alternatives of X are disjoint
  - if nullable(X), first(X) must be disjoint from follow(X) and only one alternative of X may be nullable
- For each LL(1) grammar we can build recursive-descent parser
- Each LL(1) grammar is unambiguous
- If a grammar is not LL(1), we can sometimes transform it into equivalent LL(1) grammar

Computing if a token can **follow**

first($B_1 \ldots B_p$) = {a∈Σ | $B_1 \ldots B_p$ ⇒… ⇒ aw }

follow(X) = {a∈Σ | S ⇒… ⇒ …Xa… }

There exists a derivation from the start symbol that produces a sequence of terminals and nonterminals of the form …Xa…
(the token a follows the non-terminal X)

Rule for Computing Follow

Given $X ::= YZ$ (for reachable X) $\Rightarrow \ldots X a \ldots \Rightarrow \ldots YZa \ldots$

then **first**(Z) ⊆ **follow**(Y)

and **follow**(X) ⊆ **follow**(Z)

$$\frac{a \in follow(X)}{a \in follow(Z)}$$

now take care of nullable ones as well:

For each rule $X ::= Y_1 \ldots Y_p \ldots Y_q \ldots Y_r$

follow(Y_p) should contain:

- **first**($Y_{p+1} Y_{p+2} \ldots Y_r$)
- also **follow**(X) if **nullable**($Y_{p+1} Y_{p+2} Y_r$)

Compute nullable, first, follow

stmtList ::= ε | stmt stmtList

stmt ::= assign | block

assign ::= **ID** **=** **ID** **;**

block ::= **beginof** **ID** stmtList **ID** **ends**

Is this grammar LL(1)?

Conclusion of the Solution

The grammar is not LL(1) because we have

- nullable(stmtList)
- first(stmt) ∩ follow(stmtList) = {**ID**}

- If a recursive-descent parser sees **ID**, it does not know if it should
  - finish parsing stmtList or
  - parse another stmt

Table for LL(1) Parser: Example

$\Sigma = \{EOF, (,)\}$

S ::= B **EOF**

(1)

B ::= ε | B **(**B**)**

(1) **(2)**

B ::= ε | (B)B

nullable: B

first(S) = { **(**, **EOF** }

follow(S) = {}

first(B) = { **(** }

follow(B) = { **)**, **(**, **EOF** }

empty entry:
when parsing S,
if we see) ,
report error

Parsing table:

		EOF	(	)
S	**S**	{1}	{1}	{}
B	**B**	{1}	{1,2}	{1}

parse conflict - choice ambiguity: grammar not LL(1)

1 is in entry because **(** is in follow(B)

2 is in entry because **(** is in first(B(B))

Table for LL(1) Parsing

Tells which alternative to take, given current token:

choice : Nonterminal x Token -> Set[Int]

A ::= (1) $B_1 \ldots B_p$
| (2) $C_1 \ldots C_q$
| (3) $D_1 \ldots D_r$

if t ∈ first($C_1 \ldots C_q$) add 2 to choice(A,t)

if t ∈ follow(A) add K to choice(A,t) where K is nullable

For example, when parsing A and seeing token t

choice(A,t) = {2} means: parse alternative 2 ($C_1 \ldots C_q$)

choice(A,t) = {3} means: parse alternative 3 ($D_1 \ldots D_r$)

choice(A,t) = {} means: report syntax error

choice(A,t) = {2,3} : not LL(1) grammar

General Idea when parsing nullable(A)

A ::= $B_1 \dots B_p$
| $C_1 \dots C_q$
| $D_1 \dots D_r$

def A =
if (token ∈ T1) {
$B_1 \dots B_p$
else if (token ∈ (T2 ∪ T_F)) {
$C_1 \dots C_q$
} **else if** (token ∈ T3) {
$D_1 \dots D_r$
} // no else error, just return

where:

T1 = **first**($B_1 \dots B_p$)
T2 = **first**($C_1 \dots C_q$)
T3 = **first**($D_1 \dots D_r$)
T_F = **follow**(A)

Only one of the alternatives can be nullable (here: 2nd)
T1, T2, T3, T_F should be pairwise **disjoint** sets of tokens.